# AIOps Kubernetes → Prometheus → Grafana Workflow Log

Conversation-derived record of the troubleshooting, commands, outputs, configuration changes, monitoring setup, and Prometheus metric-export workflow completed on 13 August 2026.

## 1. Initial Problem: recommendationservice CrashLoopBackOff

The starting symptom was a Kubernetes recommendationservice container repeatedly restarting. The user initially ran:

```
kubectl describe pod -l app=recommendationservice | grep -A 5 "Last State"
```

### Pasted response:

```
Last State:     Terminated
  Reason:       Error
  Exit Code:    137
  Started:      Thu, 13 Aug 2026 23:15:55 +0530
  Finished:     Thu, 13 Aug 2026 23:16:12 +0530
Ready:          False
```

Initial interpretation: exit code 137 means the process received SIGKILL. OOM was considered first, but the later Kubernetes Events output established that the immediate cause was an aggressive liveness probe.

## 2. Full Pod Description and Resource Configuration

The user then ran:

```
kubectl describe pod recommendationservice
```

Important pasted details:

```
Name: recommendationservice-d8cbbc44-vnvxs
Namespace: default
Node: aiops-cluster-worker2/172.18.0.2
Image: .../recommendationservice:v0.10.6
Port: 8080/TCP

State: Waiting
  Reason: CrashLoopBackOff

Last State: Terminated
  Reason: Error
  Exit Code: 137

Ready: False
Restart Count: 6

Limits:
  cpu:     200m
  memory:  450Mi

Requests:
  cpu:     100m
  memory:  220Mi

Liveness:  grpc <pod>:8080  delay=0s timeout=1s period=5s
Readiness: grpc <pod>:8080  delay=0s timeout=1s period=5s

Environment:
  PORT: 8080
  PRODUCT_CATALOG_SERVICE_ADDR: productcatalogservice:3550
  DISABLE_PROFILER: 1
```

The resource limit was 450 MiB, but no explicit OOMKilled reason was shown. The probe configuration stood out because liveness checking began immediately with a 1-second timeout.

# 3. Kubernetes Events Identified the Actual Cause

Command used:

```
kubectl describe pod recommendationservice-d8cbbc44-vnvxs | grep -A 20 "Events:"
```

Critical pasted Events:

```
Normal   Killing    ...  Container server failed liveness probe, will be restarted
Warning  Unhealthy  ...  Readiness probe failed: timeout: failed to connect service "10.244.2.6:8080" within 1s
Warning  Unhealthy  ...  Liveness probe failed: timeout: failed to connect service "10.244.2.6:8080" within 1s
Warning  BackOff    ...  Back-off restarting failed container server
```

Conclusion: the service was not accepting gRPC connections on port 8080 within the 1-second probe timeout during startup. Kubernetes therefore killed and restarted it. The CrashLoopBackOff was a consequence of this probe behavior.

# 4. Workflow Modification: Add a startupProbe

The Deployment was edited with kubectl edit. The startupProbe was inserted at the same indentation level as livenessProbe, not inside it.

```
kubectl edit deployment recommendationservice
```

The added configuration was:

```
startupProbe:
  failureThreshold: 12
  grpc:
    port: 8080
    service: ""
  periodSeconds: 5
  successThreshold: 1
  timeoutSeconds: 2

livenessProbe:
  failureThreshold: 3
  grpc:
    port: 8080
    service: ""
  periodSeconds: 5
  successThreshold: 1
  timeoutSeconds: 1
```

Effect: Kubernetes now allows the service time to initialize before liveness enforcement. With failureThreshold=12 and periodSeconds=5, the startup probe can tolerate roughly 60 seconds of startup attempts.

# 5. Rollout Verification

The user watched the pods with:

```
kubectl get pods -l app=recommendationservice -w
```

Pasted intermediate output:

```
NAME                                 READY   STATUS              RESTARTS      AGE
recommendationservice-6796bb74ff-v22ds   0/1     ContainerCreating   0             10s
recommendationservice-d8cbbc44-vnvxs     1/1     Running             7 (10m ago)   16m
```

The rollout was then checked:

```
kubectl rollout status deployment/recommendationservice

deployment "recommendationservice" successfully rolled out
```

Final pod status:

```
kubectl get pods -l app=recommendationservice

NAME                                 READY   STATUS    RESTARTS   AGE
recommendationservice-6796bb74ff-v22ds   1/1     Running   0          76s
```

# 6. Final Probe Verification

Command:

```
kubectl describe pod -l app=recommendationservice | grep -A 20 "Events:"
```

Pasted output showed:

```
Created container server
Started container server
Warning Unhealthy ... Startup probe failed: timeout: failed to connect service "10.244.1.7:8080" within 2s (x2)
```

This was considered healthy behavior because the startup probe failed twice during initialization, but the container was not killed. The pod reached 1/1 Running with 0 restarts. This confirmed that the startupProbe solved the CrashLoopBackOff.

# 7. Prometheus/Grafana Investigation

The next goal was to inspect the application through Prometheus/Grafana. A port-forward was initially attempted using a standard monitoring namespace:

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090
```

Initial error:

```
Error from server (NotFound): namespaces "monitoring" not found
```

The cluster was inspected:

```
kubectl get pods --all-namespaces | grep -i prometheus
kubectl get ns
kubectl get svc -A
```

Pasted namespace list showed only:

```
default
kube-node-lease
kube-public
kube-system
local-path-storage
```

Conclusion at that point: Prometheus/Grafana were not installed in this fresh cluster. The application itself was running in the default namespace.

# 8. Installing kube-prometheus-stack

The monitoring Helm repositories were added/updated and the stack was installed:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

helm install prometheus prometheus-community/kube-prometheus-stack   --namespace monitoring --create-namespace   --set alertmanager.enabled=
```

Pasted Helm result:

```
NAME: prometheus
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1

kube-prometheus-stack has been installed.
```

This created the monitoring namespace and installed Prometheus, Grafana, kube-state-metrics, node exporter, and the Prometheus Operator components.

# 9. Prometheus Connectivity and Health

Prometheus was accessed through a local port-forward. A second port-forward attempt later failed because port 9090 was already occupied. The process holding the port was identified with:

```
lsof -i :9090

COMMAND   PID    USER           FD   TYPE  DEVICE  NODE NAME
kubectl  51303  shreesh-kolhatkar   8u   IPv4  ...     TCP localhost:9090 (LISTEN)
kubectl  51303  shreesh-kolhatkar   9u   IPv6  ...     TCP ip6-localhost:9090 (LISTEN)
```

The existing port-forward was verified with:

```
curl http://localhost:9090/-/ready

Prometheus Server is Ready.
```

This established that Prometheus was reachable at localhost:9090 and that a second port-forward was unnecessary.

# 10. Prometheus 'up' Query

In the Prometheus UI, the query below was executed:

```
up
```

The pasted result contained 26 series. Examples included kubelet, Prometheus Operator, node-exporter and kube-state-metrics targets, all returning 1. A value of 1 means the target is successfully being scraped.

# 11. CPU Metric Query Verification

The exact CPU query used by the original exporter was tested directly in Prometheus:

```
sum(rate(container_cpu_usage_seconds_total{namespace="default"}[1m])) by (pod)
```

Pasted result:

```
Evaluation time
Load time: 35ms
Result series: 12

redis-cart-7ff8f4d6ff-cxw2h        0.008123783599292876
shippingservice-6df4d977f6-qq2rn   0.0026192504795318765
cartservice-5779b4b5f-7spzx        0.01565672844128803
frontend-5d77f9d6d7-nk572          0.052099835318991136
productcatalogservice-5b94cd5776-pc6jt 0.023921982681273504
currencyservice-74cd85cd6-spkgf    0.031877392077595754
paymentservice-76db4cc7fb-mn2xv    [value present in UI output]
```

This proved that Prometheus was collecting the desired container CPU metric. The CPU values represent CPU cores being consumed by each pod as calculated by rate(...).

# 12. Original export_metrics.py and Why It Reported No Data

The user shared the original exporter. Its key behavior was:

```
PROMETHEUS_URL = "http://localhost:9090"

END_TIME = datetime.utcnow()
START_TIME = END_TIME - timedelta(minutes=20)
STEP = "15s"

QUERY = 'sum(rate(container_cpu_usage_seconds_total{namespace="default"}[1m])) by (pod)'

response = requests.get(
    f"{PROMETHEUS_URL}/api/v1/query_range",
    params={
        "query": QUERY,
        "start": START_TIME.timestamp(),
        "end": END_TIME.timestamp(),
        "step": STEP,
    }
)
```

The script printed a deprecation warning for datetime.utcnow() and then reported no data. Because the exact PromQL returned 12 series when tested in Prometheus, Prometheus itself was not the root problem. The exporter was then rewritten to be more robust.

# 13. Final Exporter Workflow Modification

The revised exporter was designed to preserve the historical 20-minute query model while improving reliability. The changes were:

- Use the working CPU PromQL and explicitly exclude empty/POD container series.
- Use timezone-aware datetime.now(timezone.utc), eliminating the utcnow() deprecation warning.
- Check Prometheus /-/ready before querying.
- Use a request timeout and handle HTTP/API errors.
- Handle an empty Prometheus result cleanly.
- Convert results into a pandas DataFrame with timestamp, pod and cpu_usage columns.
- Sort data by timestamp and pod.
- Write cpu_metrics.csv containing per-pod CPU readings.
- Write total_cpu_timeseries.csv containing total CPU across pods per timestamp.
- Print basic statistics and the recorded pod names for verification.

## Recommended working query in the revised exporter:

```
sum by (pod) (
    rate(
        container_cpu_usage_seconds_total{
            namespace="default",
            container!="",
            container!="POD"
        }[1m]
    )
)
```

# 14. Final End-to-End Workflow

The completed workflow is now:

1. Kubernetes cluster runs Online Boutique microservices.
2. recommendationservice originally CrashLooped because its liveness probe was too aggressive during startup.
3. Added startupProbe to recommendationservice Deployment.
4. Rollout succeeded; recommendationservice became 1/1 Running with 0 restarts.
5. Installed kube-prometheus-stack into monitoring namespace with Helm.

6. Prometheus began scraping Kubernetes/node/application metrics.
7. Prometheus was exposed locally on localhost:9090 through kubectl port-forward.
8. Prometheus /-/ready returned "Prometheus Server is Ready."
9. up query returned 26 healthy scrape targets.
10. CPU PromQL returned 12 application-pod series.
11. export_metrics.py was rewritten to query and export the CPU time series.
12. CSV outputs are intended for later Prophet/LSTM/AIOps analysis.

# 15. Commands: Consolidated Reference

| Purpose | Command |
|---|---|
| Inspect recommendationservice | kubectl describe pod -l app=recommendationservice \| grep -A 5 "Last State" |
| Full pod description | kubectl describe pod recommendationservice |
| Inspect Events | kubectl describe pod recommendationservice-d8cbbc44-vnvxs \| grep -A 20 "Events:" |
| Edit Deployment | kubectl edit deployment recommendationservice |
| Watch pod rollout | kubectl get pods -l app=recommendationservice -w |
| Check rollout | kubectl rollout status deployment/recommendationservice |
| Check final pod | kubectl get pods -l app=recommendationservice |
| Check final Events | kubectl describe pod -l app=recommendationservice \| grep -A 20 "Events:" |
| List all Prometheus pods | kubectl get pods --all-namespaces \| grep -i prometheus |
| List namespaces | kubectl get ns |
| List services | kubectl get svc -A |
| Add Prometheus Helm repo | helm repo add prometheus-community https://prometheus-community.github.io/helm-charts |
| Add Grafana Helm repo | helm repo add grafana https://grafana.github.io/helm-charts |
| Update Helm repos | helm repo update |
| Install monitoring stack | helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace --set alertmanager.enabled=false --set grafana.resources.requests.memory=128Mi --set prometheus.prometheusSpec.resources.requests.memory=256Mi |
| Port-forward Prometheus | kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090 |
| Find port owner | lsof -i :9090 |
| Test Prometheus readiness | curl http://localhost:9090/-/ready |
| Prometheus target health | up |
| CPU query | sum(rate(container_cpu_usage_seconds_total{namespace="default"}[1m])) by (pod) |
| Check exporter file | sed -n '1,250p' export_metrics.py |
| Run exporter | python3 export_metrics.py |
| Inspect generated CSVs | ls -lh cpu_metrics.csv total_cpu_timeseries.csv |
| Preview CPU CSV | head cpu_metrics.csv |

# 16. Current Known-Good State

Kubernetes:
  recommendationservice: 1/1 Running
  Restarts: 0

Monitoring:

```
  monitoring namespace: exists
  kube-prometheus-stack: deployed
  Prometheus: reachable at localhost:9090
  Prometheus readiness: "Prometheus Server is Ready."
  up query: 26 result series
  CPU query: 12 result series

Data collection:
  Prometheus CPU metrics are available.
  export_metrics.py has been redesigned to export:
    - cpu_metrics.csv
    - total_cpu_timeseries.csv
```

# 17. Important Operational Notes

• Keep the Prometheus port-forward terminal running while the exporter uses http://localhost:9090.

• If port 9090 is already occupied, do not start another port-forward; use lsof -i :9090 to identify the existing kubectl process.

• The startupProbe is intentional: it prevents the liveness probe from killing recommendationservice before the gRPC server is ready.

• Exit code 137 was observed, but the decisive Kubernetes Events showed liveness-probe failures rather than an OOMKilled reason.

• For forecasting/AIOps data collection, generate workload/load-test traffic during the metric collection window so the recorded time series contains meaningful variation.

End of conversation-derived workflow log.